

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Hemanth Kumar R
24011101035
B.Tech AI&DS 'A'

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

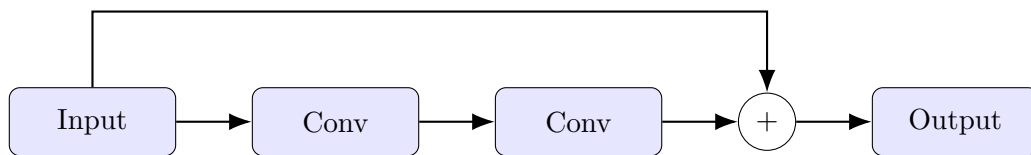


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

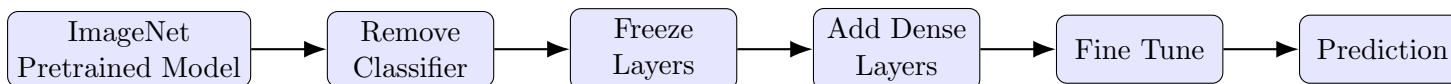


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range $[0,1]$.
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10–20
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

16. Hyperparameter Study

Compare the performance using different settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

17. Source Code

The complete implementation for the comparative study of deep convolutional neural network architectures using the CIFAR-10 dataset is available in the GitHub repository below. The repository contains the Jupyter Notebook with the implementations of LeNet-5, AlexNet, VGG16, GoogleNet, and ResNet50, along with the transfer learning and fine-tuning experiments, hyperparameter study, evaluation metrics, generated plots, confusion matrix, and misclassified images.

<https://github.com/AI-MANIAC007/Hands-On-Deep-Learning>

18. Mandatory Plots



Figure 3: Sample CIFAR-10 Images

The figure shows ten sample images from the CIFAR-10 dataset with the corresponding class labels. The samples represent Frog, Truck, Truck, Deer, Automobile, Automobile, Bird, Horse, Ship, and Cat. Since CIFAR-10 images have a resolution of only 32×32 pixels, the objects contain limited fine-grained visual information. Some objects also occupy only a small part of the image or appear with complex

backgrounds. Therefore, the model must learn useful features from relatively low-resolution images, which makes some classes difficult to distinguish.

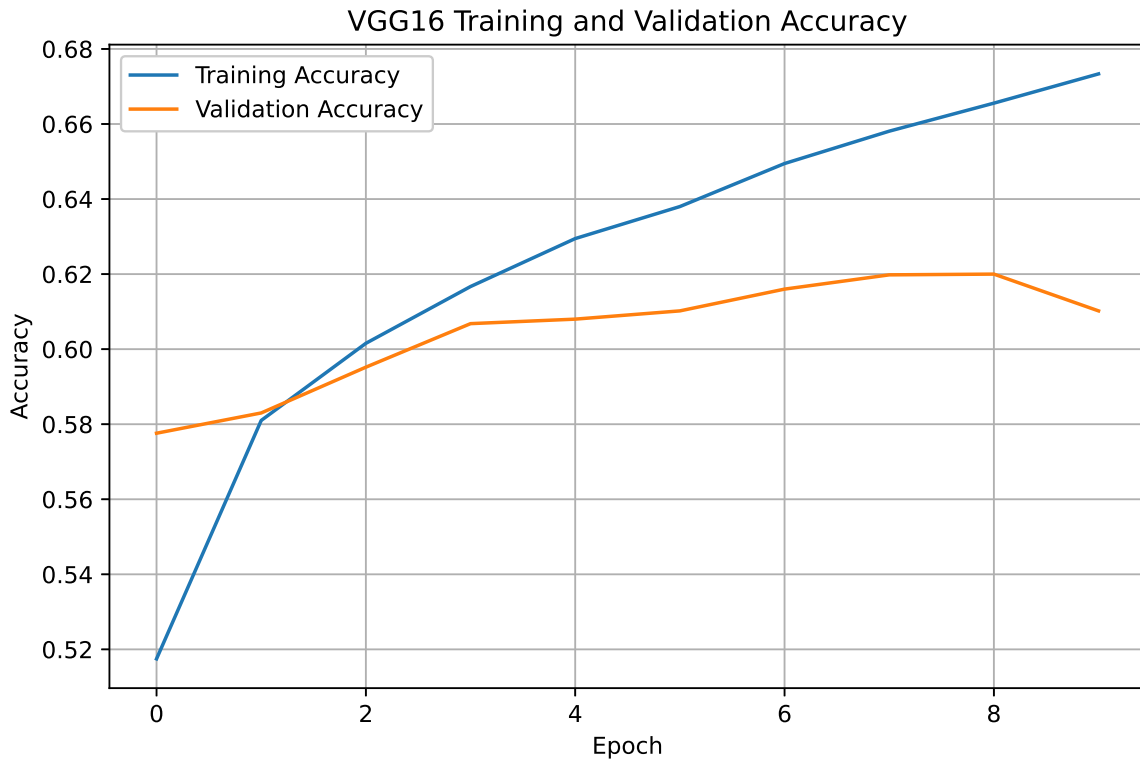


Figure 4: VGG16 Training and Validation Accuracy

The accuracy of training is increasing from approximately 51.97% during the first epoch to 66.94% during the tenth epoch, implying that the network is gradually learning the features of the data used for training. The accuracy of validation is also increasing at the initial epochs to reach approximately 62.30% at the ninth epoch, then falls to approximately 60.94% at the final epoch. This shows that while training accuracy is continuously increasing, the accuracy of validation is no longer increasing, leading to an increasing gap between the two.

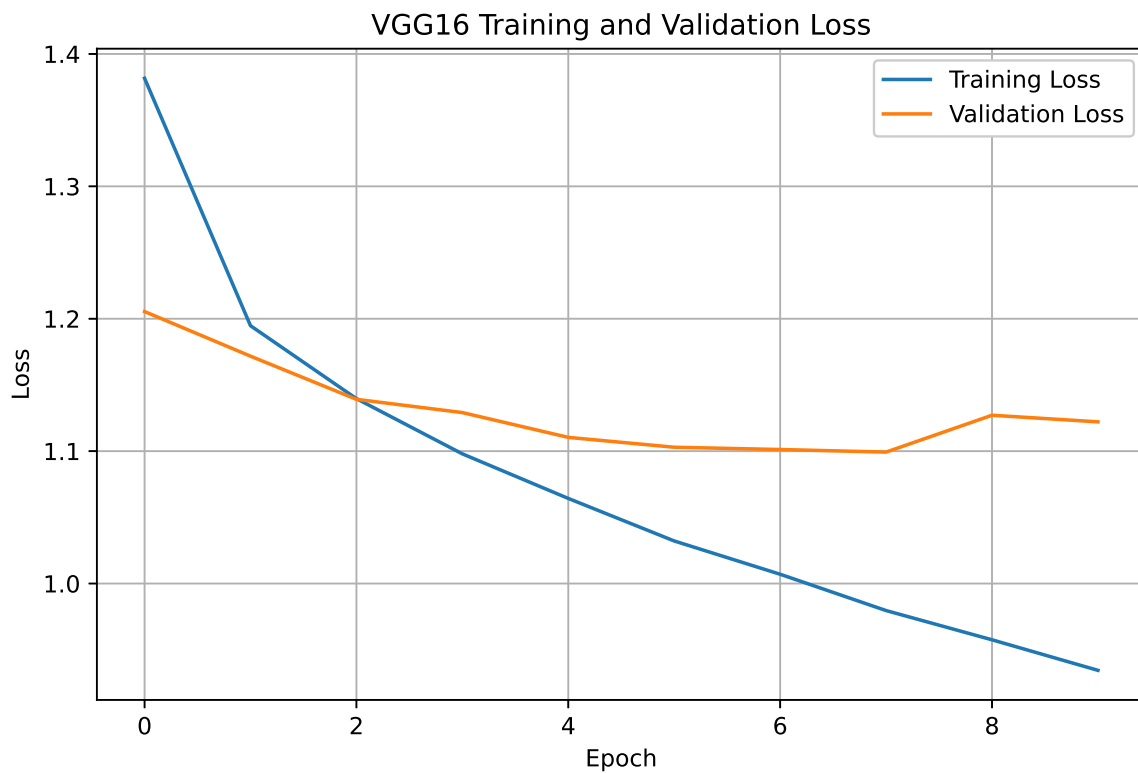


Figure 5: VGG16 Training and Validation Loss

Training Loss decreases continuously from 1.38 to 0.94, which indicates that the model is improving its ability to fit the training data. Validation Loss also decreases in the beginning epochs; however, it does not improve after a certain point and starts fluctuating. Furthermore, the gap between training and validation accuracy increases, which means that the model becomes overfitted after the peak of the validation performance.

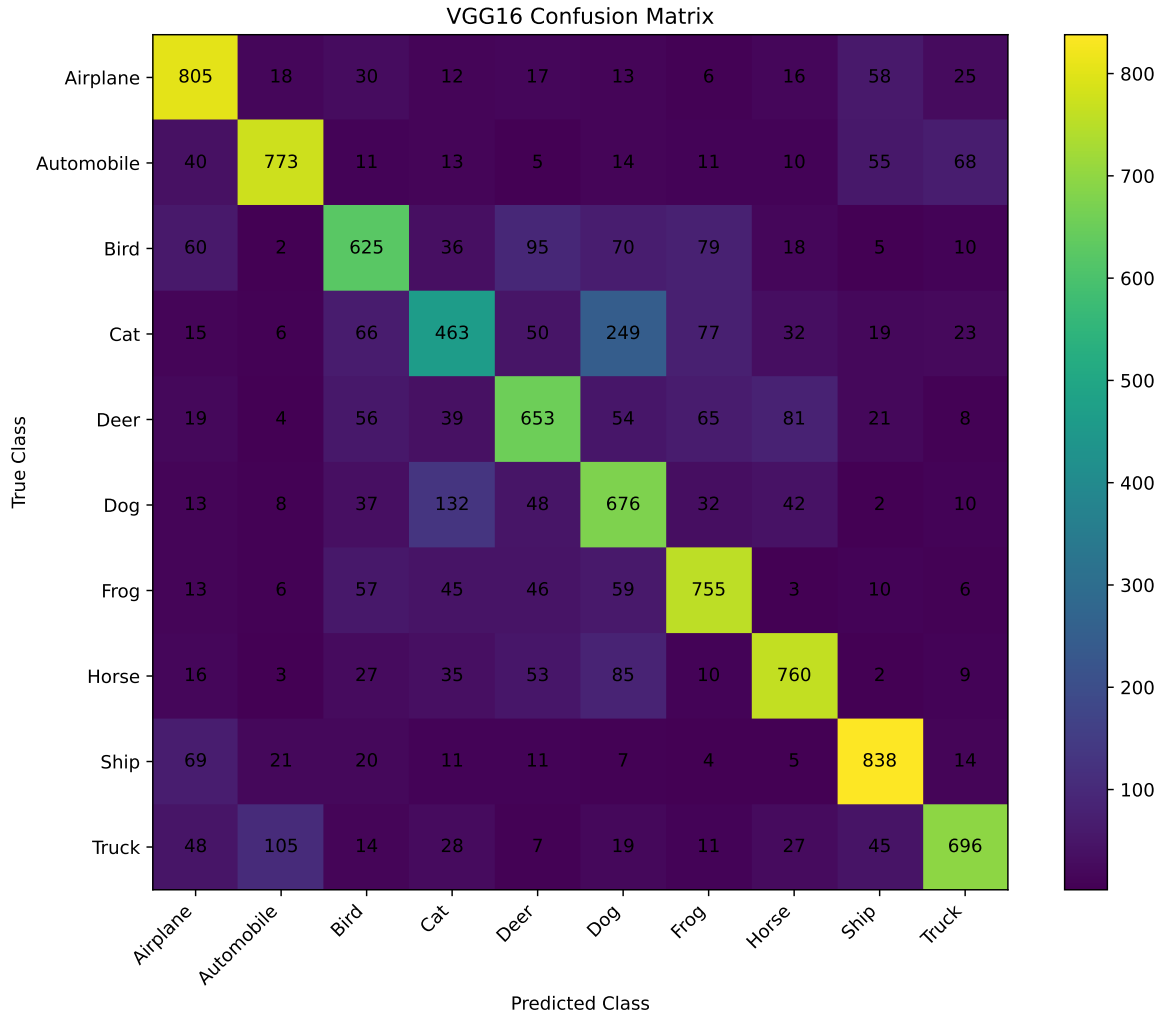


Figure 6: VGG16 Confusion Matrix

The confusion matrix provides a class-wise view of the VGG16 predictions. The diagonal elements represent correctly classified images, while the off-diagonal elements represent misclassifications. The model correctly classifies a large number of Ship, Airplane, Automobile, Horse, Frog, and Truck images. However, noticeable confusion occurs between visually similar classes. In particular, 249 Cat images are predicted as Dog and 132 Dog images are predicted as Cat. Bird is also confused with Deer and Frog, while Truck is frequently confused with Automobile. These errors can be explained by the low 32×32 image resolution and the similarity in shape, texture, and background between some CIFAR-10 classes.

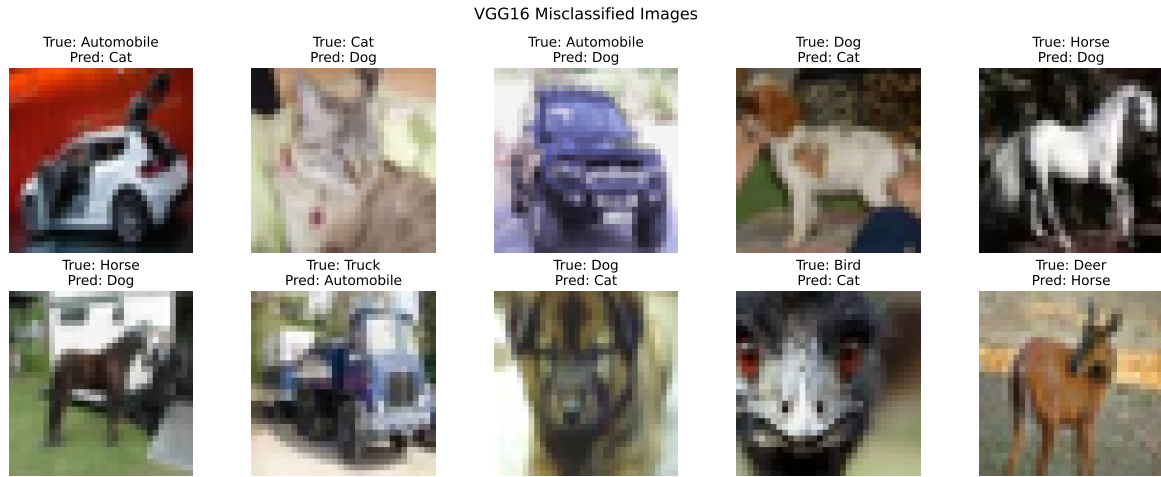


Figure 7: VGG16 Misclassified Images

These examples help to visualize the errors made in the confusion matrix. Examples like: Automobile classified as Cat, Cat classified as Dog, Automobile classified as Dog, Dog classified as Cat, Horse classified as Dog, Truck classified as Automobile, Bird classified as Cat, and Deer classified as Horse have been shown in the above figure. The errors occur mostly among those classes which have similar visual features. The small resolution of 32×32 pixels has also eliminated any finer features that might be used to classify objects. Hence, the misclassified images prove that the model has learned features but cannot separate them properly.

19. Results

19.1 Performance Metrics

Metric	Value
Training Accuracy	66.94%
Testing Accuracy	70.44%
Precision	0.7066
Recall	0.7044
F1-score	0.7035
Training Time	133.37 s (2.22 min)
Total Parameters	14,781,642

19.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5	92,246	53.00	57.70 s
AlexNet	36,925,322	10.00	315.10 s
VGG16	14,781,642	60.75	133.37 s
GoogleNet	4,176,506	73.04	449.70 s
ResNet50	23,608,202	59.93	465.92 s

20. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet was an important achievement for the following reasons: it demonstrated the fact that with the help of large amounts of data it is possible to train the neural network and achieve high quality image classification results. AlexNet used several key techniques including ReLU activation, Dropout, GPU training and data augmentation.

2. **Why does VGG16 use only 3×3 convolution filters?**

VGG16 uses small 3×3 filters repeatedly instead of using larger filters. Stacking multiple 3×3 convolutions gives a larger effective receptive field while using fewer parameters than a single large filter. It also allows the network to become deeper and learn more complex features through multiple nonlinear activation layers.

3. **Explain the advantages of the Inception module.**

The Inception module performs different operations in parallel, such as 1×1 , 3×3 , 5×5 convolutions and pooling. This allows the network to capture features at different scales at the same time. The 1×1 convolutions also reduce the number of parameters before applying larger convolutions, which reduces computational cost. Therefore, GoogleNet can be deep while still being computationally efficient.

4. **What is the purpose of residual learning?**

Residual learning is used in ResNet to make very deep networks easier to train. Instead of directly learning the complete mapping $H(x)$, the network learns the residual mapping

$$F(x) = H(x) - x$$

and then adds the original input through a skip connection:

$$H(x) = F(x) + x.$$

The skip connection allows gradients to flow more easily through the network and helps reduce the vanishing gradient problem.

5. **Differentiate LeNet and ResNet.**

LeNet is an early and relatively small CNN designed mainly for handwritten digit recognition. It has a small number of layers and parameters and uses convolution and average pooling layers. ResNet is a much deeper architecture that uses residual or skip connections. LeNet is simple and computationally inexpensive, whereas ResNet is designed to train very deep networks and achieve high classification accuracy.

6. **What is Transfer Learning?**

Transfer learning is the process of using a model that has already been trained on a large dataset, such as ImageNet, and adapting it to a new task. Instead of training the CNN completely from scratch, the pretrained model provides useful general image features. In our experiment, we load pretrained VGG16 weights, remove its original classifier, freeze the convolutional base, add new classification layers and train them for CIFAR-10.

7. **Why is fine tuning required?**

The features learned by a pretrained model are general, but they may not be perfectly suited to the new dataset. Fine tuning allows some of the pretrained convolutional layers to learn features that are more specific to the new task. In our experiment, we unfreeze the last convolutional block of VGG16 and train it along with the new classifier using a smaller learning rate.

8. **Explain the difference between dilated convolution and transpose convolution.**

The dilated convolution enlarges the receptive field of the convolution without adding more parameters. This is achieved by creating holes in between the elements of the kernel and becomes handy when we need to cover a large area within an image.

The transpose convolution is primarily applied for enlarging the spatial dimensions of a feature map. The operation does up-sampling in a learnable way. This is commonly seen in tasks such as image generation, autoencoders, and semantic segmentation.

9. **Why do pretrained models converge faster?**

Pretrained models have already learned useful features such as edges, textures and shapes from a large dataset like ImageNet. Therefore, the model does not have to learn these basic features from the beginning. When we transfer these learned features to a new task, the model starts from

a better set of weights and usually reaches a good solution faster than a model trained completely from scratch.

10. Compare the computational complexity of LeNet and ResNet.

The LeNet has an architecture that is small in size having only a few convolutional layers and fully connected layers. This gives the LeNet a smaller number of parameters making the computations simpler. ResNet, on the other hand, has many convolutional layers, making it deep and having a lot of parameters. Therefore, the ResNet requires a lot of computing power but with residual connections it manages to train very deep networks.

21. Additional Exercises

21.1 Transfer Learning using VGG16

Transfer learning was implemented using VGG16 pretrained on the ImageNet dataset. The original classification head was removed and the convolutional base was initially frozen. A Global Average Pooling layer, a Dense layer with 128 ReLU units, and a 10-class Softmax output layer were added for CIFAR-10 classification.

The model was trained using the Adam optimizer with a learning rate of 0.001, batch size of 32, and 10 epochs. The model achieved a test accuracy of **60.75%** before fine-tuning.

The pretrained VGG16 model was able to transfer general visual features learned from ImageNet to the CIFAR-10 classification task. Freezing the convolutional base reduces the number of parameters that need to be learned and allows the newly added classifier to adapt to the ten CIFAR-10 classes. Although the initial accuracy was moderate, the pretrained features provided a useful starting point for further adaptation through fine-tuning.

21.2 Repeat the Experiment using ResNet50

The experiment was repeated using ResNet50 with ImageNet-pretrained weights. The pretrained convolutional base was frozen and a new classification head consisting of Global Average Pooling, a Dense layer with 128 ReLU units, and a 10-class Softmax output was added.

The model was trained for 10 epochs using Adam with a learning rate of 0.001 and batch size of 32. The model contained **23,851,274** parameters and achieved a test accuracy of **64.06%**. The training time was approximately **155.28 seconds**.

ResNet50 achieved a higher accuracy than the initial frozen VGG16 model in this experiment. The residual connections in ResNet allow information and gradients to flow more effectively through a deep network. Since the convolutional layers were pretrained on ImageNet and kept frozen, the model could use already learned visual representations while only the new classification head was adapted to CIFAR-10.

21.3 Comparison of Adam and SGD Optimizers

Adam and SGD were compared using the same VGG16 architecture, pretrained weights, frozen convolutional base, learning rate of 0.001, batch size of 32, and 10 training epochs. The optimizer was the main variable changed between the two experiments.

Optimizer	Test Accuracy (%)	Training Time (s)
Adam	61.10	136.44
SGD	47.99	141.48

Adam achieved a considerably higher test accuracy of 61.10% compared with 47.99% obtained using SGD under the same experimental conditions. Adam maintains adaptive estimates of the first and second moments of the gradients, allowing it to adjust the effective learning rate for individual parameters. SGD uses a simpler gradient update based on the learning rate. In this experiment, Adam therefore optimized the newly added classifier more effectively than SGD.

21.4 Frozen Layers versus Fine-Tuning

The effect of fine-tuning was studied using the pretrained VGG16 model. Initially, the complete convolutional base was frozen and only the newly added classification layers were trained. The model achieved a test accuracy of **60.75%**.

After this stage, the last convolutional block was unfrozen and the model was fine-tuned using a smaller learning rate of 0.00001 for five additional epochs. After fine-tuning, the test accuracy increased to **70.44%**.

Training Method	Test Accuracy (%)
Frozen Convolutional Base	60.75
Fine-Tuning	70.44
Improvement	9.69

Freezing the convolutional base preserves the general features learned from ImageNet, but these features are not fully adapted to CIFAR-10. During fine-tuning, the last convolutional block was allowed to update its weights, enabling the higher-level feature representations to become more suitable for the new dataset. The improvement from 60.75% to 70.44% shows that fine-tuning provided better adaptation to the CIFAR-10 task. A small learning rate was used during fine-tuning to avoid drastically changing the useful pretrained weights.

21.5 Precision, Recall and F1-score

The VGG16 model was further evaluated using precision, recall, and F1-score in addition to accuracy. Weighted averaging was used to account for the contribution of all CIFAR-10 classes.

Metric	Value
Precision	0.7066
Recall	0.7044
F1-score	0.7035

The precision of 0.7066 indicates that approximately 70.66% of the weighted predicted positive classifications were correct, while the recall of 0.7044 indicates that approximately 70.44% of the weighted actual instances were correctly identified. The F1-score of 0.7035 provides a single measure that balances precision and recall. The close values of precision and recall indicate that the model does not show a very large imbalance between these two aspects of classification performance.

21.6 Comparison of LeNet-5, AlexNet and ResNet50

The performance of LeNet-5, AlexNet, and ResNet50 was compared using the results obtained during the CNN architecture experiments.

Model	Parameters	Accuracy (%)	Training Time (s)
LeNet-5	92,246	53.00	57.70
AlexNet	36,925,322	10.00	315.10
ResNet50	23,608,202	59.93	465.92

LeNet-5 has by far the smallest number of parameters and the shortest training time, but its simpler architecture limits its ability to learn complex visual representations. AlexNet contains a much larger number of parameters and consequently requires more computation. However, its accuracy in this particular implementation is only 10%, indicating that the model did not learn the CIFAR-10 task effectively under the selected training settings.

ResNet50 achieved the highest accuracy among the three models at 59.93%. Although it required the longest training time, its residual connections make it possible to train a much deeper network effectively. Therefore, the comparison shows that a larger and deeper architecture does not automatically guarantee better performance; the architecture, optimization strategy, preprocessing, and training configuration all influence the final result.

22. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>